

IL PRINCIPIO DI SOSTITUZIONE DI LISKOV

Fasi della progettazione e dello sviluppo

Idealmente, la **progettazione e lo sviluppo** di un programma Java dovrebbe attraversare le seguenti **fasi**

1. **Definizione** della **gerarchia di classi e interfacce** da realizzare
2. **Identificazione** dei **membri pubblici** di ogni classe
3. **Definizione** delle **SPECIFICHE** di ogni metodo pubblico (condizioni sui parametri, sul risultato e comportamento atteso del metodo)
4. **Implementazione** di **programmi di testing** delle singole classi sulla base di quanto definito (membri pubblici e specifiche)
5. **Definizione** dei **membri privati** seguendo il **principio di incapsulamento**
6. **Implementazione** del **codice delle classi**, da verificare con i test già sviluppati

WRITE TESTS
FIRST!

Definire le specifiche

Definire le specifiche di una classe significa **esprimere in modo non ambiguo il comportamento atteso** dei suoi metodi pubblici

Come?

- ▶ Aggiungendo opportuni **commenti** al codice
- ▶ Esprimendo delle **condizioni** sui parametri e sulle variabili che siano **verificabili** o **testabili**, ad esempio, con `assert`

Specifiche: esempio IntSet

Esempio di specifica con commenti

```
// OVERVIEW: un IntSet è un insieme di interi
public class IntSet {

    public IntSet(int capacity) {
        // REQUIRES: capacity non negativo
        // EFFECTS: crea un insieme vuoto che può ospitare capacity elementi
    }

    public boolean add(int elem) throws FullSetException {
        // REQUIRES: numero di elementi contenuti nell'insieme minore di capienza
        //             altrimenti eccezione
        // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,
        //             altrimenti restituisce false
    }

    public boolean contains(int elem) {
        // REQUIRES:
        // EFFECTS: restituisce true se elem p è presente nell'insieme, false altrimenti
    }
}
```

// REQUIRES:
equivale alla condizione true
(non ci sono vincoli)

Specifiche: esempio IntSet

Esempio di specifica con commenti

```
// OVERVIEW: un IntSet è un insieme di interi
public class IntSet {

    public IntSet(int capacity) {
        // REQUIRES: capacity non negativo
        // EFFECTS: crea un insieme vuoto che può ospitare
    }

    public boolean add(int elem) throws FullSetException {
        // REQUIRES: numero di elementi contenuti nell'insieme < capacity
        //                altrimenti eccezione
        // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,
        //                altrimenti restituisce false
    }

    public boolean contains(int elem) {
        // REQUIRES:
        // EFFECTS: restituisce true se elem è presente nell'insieme, false altrimenti
    }
}
```

Può sollevare un'eccezione:

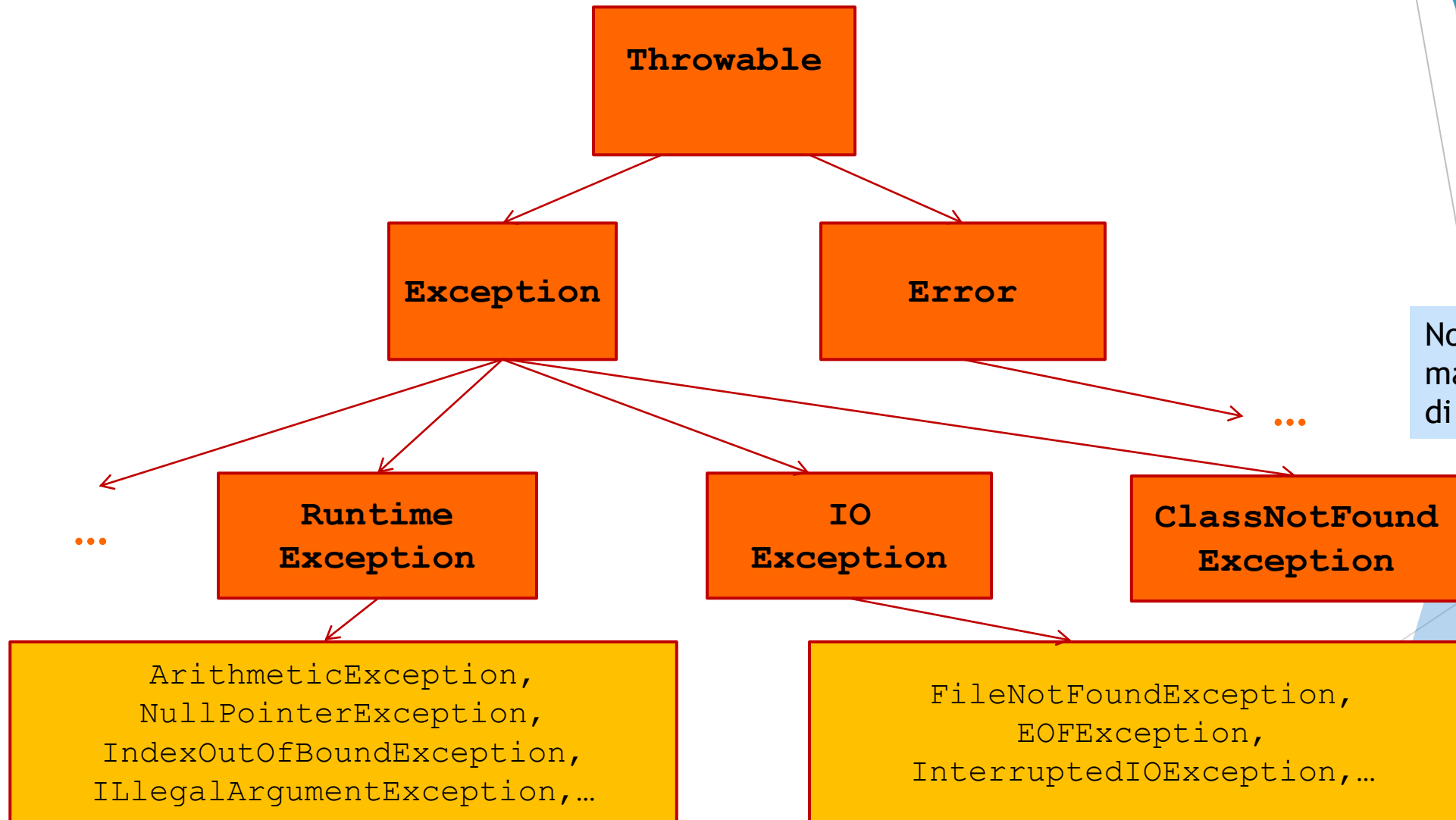
- funzionamento delle eccezioni con **throw** e **try/catch** simile a JavaScript
- Le eccezioni in Java possono essere **checked** o **unchecked**
- I metodi che sollevano eccezioni **checked** le devono gestire o lo devono dichiarare nell'intestazione (con **throws**): il chiamante è in quel caso obbligato dal compilatore a controllarle con **try/catch**

// REQUIRES:
equivale alla condizione true
(non ci sono vincoli)

Parentesi sulle eccezioni in Java

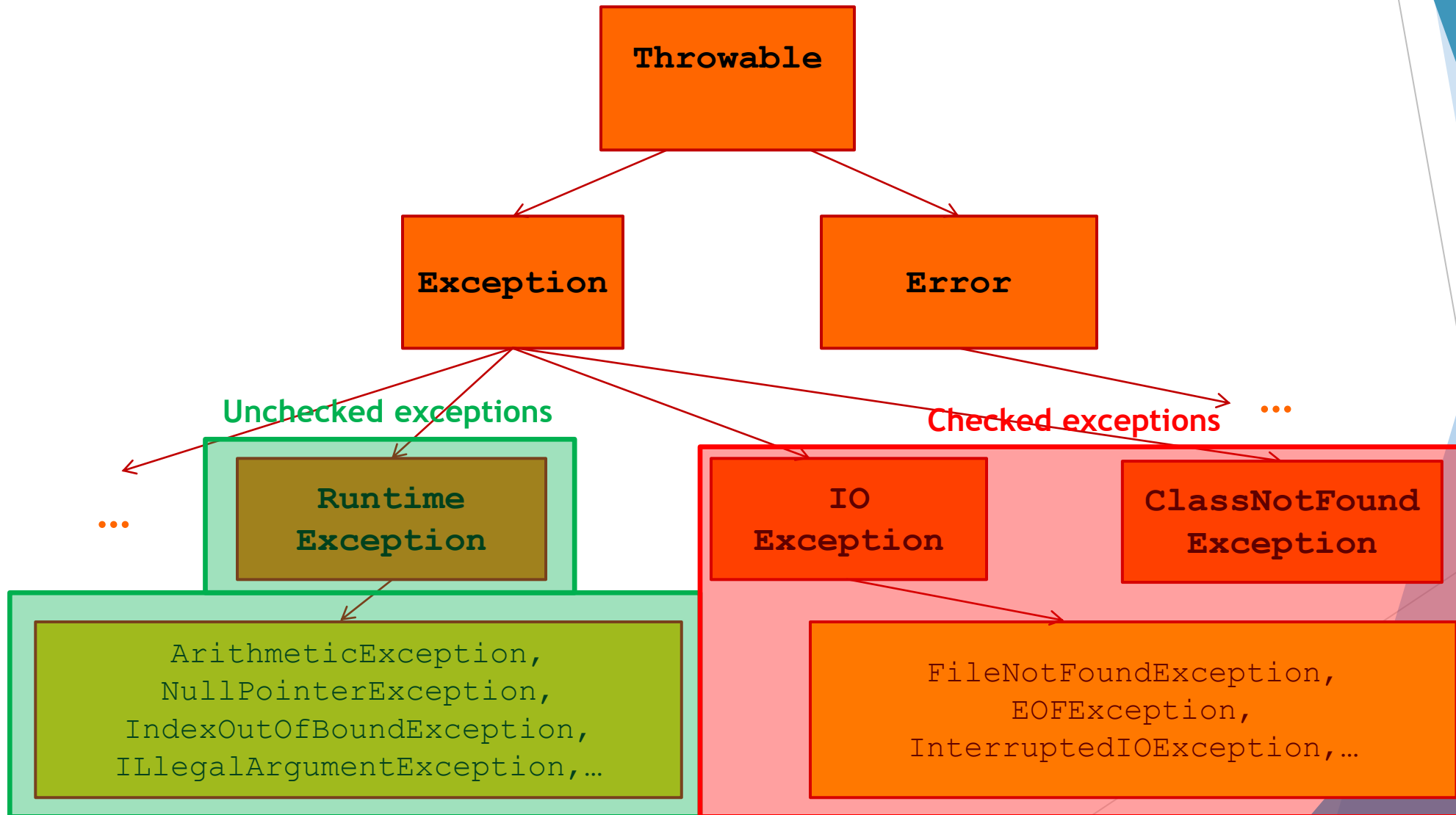
- ▶ Le **eccezioni** sono situazioni di errore o eventi imprevisti che interrompono il normale flusso di esecuzione
- ▶ Le eccezioni in Java possono essere
 - ▶ **checked** (estendono Exception, ma non includono le sottoclassi di RuntimeException), che **devono** essere controllate, o
 - ▶ **unchecked** (estendono RuntimeException) che **non devono** essere controllate. Spesso riflettono errori logici, ad es. il tentativo di accedere a un elemento di un array al di fuori dei suoi limiti (IndexOutOfBoundsException)

Le eccezioni in Java: gerarchia



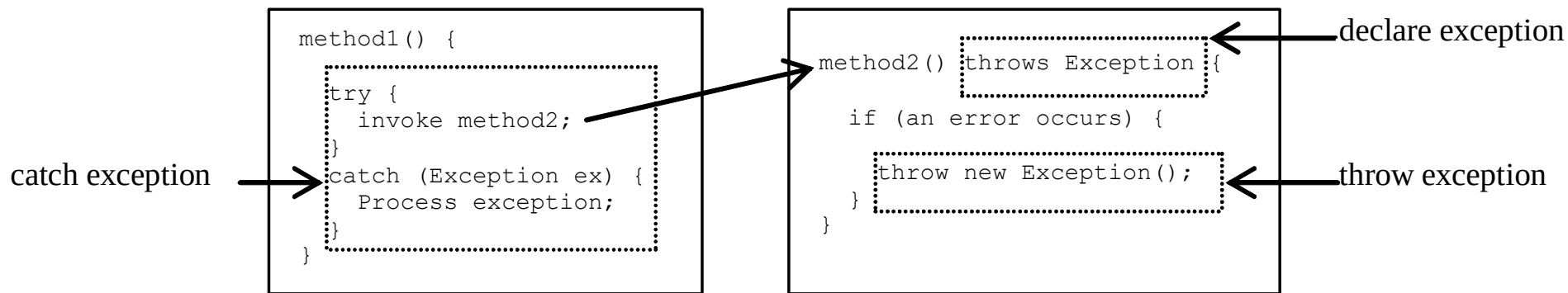
Non ci sono sottoclassi
ma se ne possono creare
di specializzate

Le eccezioni in Java: gerarchia



Le eccezioni checked

- ▶ Le **eccezioni checked** devono (lo richiede esplicitamente il compilatore) essere
 - ▶ **gestite** con **try/catch**, oppure
 - ▶ **dichiarate** nell'intestazione del metodo con **throws**



- ▶ Un metodo può scegliere di gestire direttamente un'eccezione oppure dichiararla per farla gestire dal chiamante
- ▶ La dichiarazione con **throws** è obbligatoria, solo se il metodo **non gestisce** l'eccezione e la "propaga" al chiamante

Specifiche: esempio `IntSet`

Java mette a disposizione anche una sintassi apposita per le specifiche tramite commenti (JavaDoc)

...

/**

* Aggiunge un elemento all'insieme

* @param elem valore intero

* @return true se l'elemento viene aggiunto, false se già presente

* @throws FullSetException se l'insieme è pieno

***/**

public boolean add(int elem) throws FullSetException {

...

}

...

Specifiche: esempio IntSet

Java
spec

Da questi commenti si può generare una documentazione della classe in formato HTML tramite il tool **JavaDoc** (incluso nel JDK)

```
javadoc IntSet.java
```

...

add

```
public boolean add(int elem)  
    throws FullSetException
```

Aggiunge un elemento all'insieme

Parameters:

elem - valore intero

Returns:

true se l'inserimento viene aggiunto, false se già presente

Throws:

FullSetException - se l'insieme è pieno

nte

...

Precondizioni e postcondizioni

Per ogni metodo, le specifiche indicano:

- ▶ **PRECONDIZIONI** (**//REQUIRES**): condizioni sui parametri e sul valore delle variabili che devono essere **soddisfatte prima dell'esecuzione** del metodo
- ▶ **POSTCONDIZIONI** (**//EFFECTS**): condizioni sul risultato e sul valore delle variabili che devono essere **soddisfatte dopo l'esecuzione** del metodo, **assumendo** che **le precondizioni fossero vere** all'inizio

Precondizioni e postcondizioni

...

```
public boolean add(int elem) throws FullSetException {  
    // REQUIRES: numero di elementi contenuti nell'insieme minore di capienza  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
}
```

Formalmente:

$\{P\} m \{Q\}$

quando la precondizione P è soddisfatta,
dopo l'esecuzione del metodo deve essere
vera la postcondizione Q

Specifica: dati P e Q , determinare m , tale
che $\{P\} m \{Q\}$, significa scrivere un
programma che realizza quanto specificato
dalle pre e postcondizione data

Se all'inizio è vera la **precondizione**

numero elementi < capienza

alla fine deve essere vera la **postcondizione**

risultato == (elem $\notin$ insieme)

Dalla specifica al tester

Sulla base della sola specifica possiamo già scrivere un **tester** della classe IntSet

```
public class Tester {  
  
    public static void main (String[] args) {  
  
        IntSet is = new IntSet(10);  
  
        boolean t1 = is.add(5); // TEST 1  
        if (t1) System.out.println("TEST 1 [add 5] SUPERATO");  
        else System.out.println("TEST 1 [add 5] FALLITO");  
  
        boolean t2 = is.contains(5); // TEST 2  
        if (t2) System.out.println("TEST 2 [contains 5] SUPERATO");  
        else System.out.println("TEST 2 [contains 5] FALLITO");  
  
        ...  
  
    }  
}
```

Collauda la classe controllando se i metodi implementati rispettano la **relazione tra precondizioni e postcondizioni** sugli argomenti più frequenti e su quelli che rappresentano **casi limite** (ad es. size=0)

Dalla specifica all'implementazione

Ecco un'implementazione di IntSet che soddisfa le specifiche

```
// OVERVIEW: un IntSet è un insieme di interi
public class IntSet {

    protected int[] a; // RAPPRESENTAZIONE: ARRAY DI INTERI + SIZE
    protected int size;

    public IntSet(int capacity) {
        // REQUIRES: capacity non negativo
        // EFFECTS: crea un insieme vuoto che può ospitare capacity elementi
        a = new int[capacity]; // SE capacity<0, SOLLEVA NegativeArraySizeException
        this.size = 0;
    }

    ... (SEGUE)
```

unchecked

Dalla specifica all'implementazione

... (SEGUE)

```
public boolean add(int elem) throws FullSetException {  
    // REQUIRES: numero di elementi contenuti nell'insieme minore di capienza  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
    if (size>=a.length) throw new FullSetException();  
    else if (contains(elem)) return false;  
    else { a[size]== elem; size++; return true; }  
}  
  
public boolean contains(int elem) {  
    // REQUIRES:  
    // EFFECTS: restituisce true se elem p è presente nell'insieme, false altrimenti  
    for (int i=0; i<size; i++) {  
        if (a[i]==elem) return true;  
    }  
    return false;  
}  
}
```


Verifica delle specifiche

I **test** consentono di **identificare errori** di programmazione (rispetto alle specifiche)

- ▶ **Non sono esaustivi** (in generale non si possono testare tutti i casi possibili)
- ▶ Esistono strumenti di **generazione automatica di test** (dalle specifiche) che consentono di aumentare la copertura dei casi da provare

Esistono metodi di **analisi statica** e **model checking** che consentono di **verificare** (dimostrare) in modo automatico la correttezza del codice rispetto alle specifiche

- ▶ Necessari in casi di programmi eseguiti in contesti **safety-critical** e di **cybersecurity**
- ▶ Esempi:
 - ▶ **Infer** (<https://fbinfer.com/>) analizzatore statico per Java/C/C++ (sviluppato da **Facebook**)
 - ▶ **PathFinder** (<https://github.com/javapathfinder/jpf-core/wiki>) model checker per Java (sviluppato alla **Nasa**)
- ▶ Metodi approfonditi nei **corsi della magistrale...**

Principio di Sostituzione di Liskov

(Liskov Substitution Principle - LSP)



L'esistenza di una specifica consente di ragionare sul **rapporto tra superclasse e sottoclasse** da un **punto di vista comportamentale**

Principio di Sostituzione di Liskov (LSP)

Un oggetto di un **sottotipo** può sostituire un oggetto del **supertipo** **senza influire sul comportamento** dei programmi che usano il supertipo

La sottoclasse deve soddisfare le specifiche della superclasse per poter essere utilizzata al suo posto

Principio di Sostituzione di Liskov

(Liskov Substitution Principle - LSP)

Esempio: FlexIntSet sottoclasse di IntSet per insiemi ridimensionabili

```
// OVERVIEW: un FlexIntSet è un insieme di interi ridimensionabile
public class FlexIntSet extends IntSet {

    // protected int[] a; // EREDITATI (STESSA RAPPRESENTAZIONE)
    // protected int size;

    public FlexIntSet(int capacity) {
        super(capacity); // SE capacity<0, SOLLEVA NegativeArraySizeException
    }

    ... (SEGUE)
```

Principio di Sostituzione di Liskov

... (SEGUE)

```
public boolean add(int elem) {
```

```
// REQUIRES:
```

niente più vincoli, data la possibilità di ridimensionare

```
// EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
//          altrimenti restituisce false
```

Il metodo contains viene ereditato

```
if (contains(elem)) {return false;}
```

// se l'elemento è già presente

```
if (size >= a.length) {
```

// se l'array è già pieno

```
    int[] tmp = new int[size + 10];
```

// crea un nuovo array più grande

```
    for (int i = 0; i < size; i++)
```

// copia gli elementi

```
        tmp[i] = a[i];
```

```
    a = tmp;
```

// aggiorna il riferimento

```
}
```

```
a[size] = elem;
```

// aggiunge l'elemento

```
size++;
```

```
return true;
```

```
} ...
```

Principio di Sostituzione di Liskov

(Liskov Substitution Principle - LSP)

Confrontiamo le specifiche del metodo add

► Nella **superclasse** `IntSet`:

```
public boolean add(int elem) throws FullSetException {  
    // REQUIRES: numero di elementi contenuti nell'insieme minore di capienza  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
}
```

► Nella **sottoclasse** `FlexIntSet`:

```
public boolean add(int elem) {  
    // REQUIRES:  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
}
```

Il metodo della sottoclasse

- ha precondizioni più deboli ($A \Rightarrow \text{true}$, per ogni A)
- funziona in più casi rispetto a quello della superclasse
- non solleva eccezioni

Principio di Sostituzione di Liskov

(Liskov Substitution Principle - LSP)

È facile vedere che gli oggetti della classe FlexIntSet sono **sostituibili** (secondo la definizione di Liskov) agli oggetti della classe IntSet

- **In qualunque contesto che faccia un uso appropriato di IntSet** (ad es. che chiami i suoi metodi in modo appropriato rispetto alle precondizioni) **si può usare un FlexIntSet ottenendo esattamente lo stesso comportamento**

```
IntSet set = new IntSet(10);    // esempio di contesto in cui si usa IntSet
for (int i=0; i<10; i++)        // inserisce 1,1,2,2,3,3,4,4,5,5
    set.add(i/2);
System.out.println(set.contains(3)); // true
System.out.println(set.contains(6)); // false
```

Principio di Sostituzione di Liskov

(Liskov Substitution Principle - LSP)

È facile vedere che gli oggetti della classe FlexIntSet sono **sostituibili** (secondo la definizione di Liskov) agli oggetti della classe IntSet

- **In qualunque contesto che faccia es. che chiami i suoi metodi in precondizioni) si può usare un stesso comportamento**

Il comportamento del programma non cambia!

- questo non sarebbe vero se il contesto cercasse di inserire 11 elementi diversi, ma in quel caso starebbe violando le precondizioni di IntSet (e non quelle di FlexIntSet)
- la sottoclasse **soddisfa** il principio di sostituzione di Liskov

```
IntSet set = new FlexIntSet(10); // sostituzione IntSet -> FlexIntSet
for (int i=0; i<10; i++)          // inserisce 1,1,2,2,3,3,4,4,5,5
    set.add(i/2);
System.out.println(set.contains(3)); // true
System.out.println(set.contains(6)); // false
```



Altro esempio: Rettangolo e Quadrato

Consideriamo una classe **Rectangle** con le seguenti **specifiche**

```
public class Rectangle {  
  
    public void setWidth(int w) {  
        // REQUIRES: w non negativo  
        // EFFECTS: imposta la larghezza a w  
    }  
  
    public void setHeight(int h) {  
        // REQUIRES: h non negativo  
        // EFFECTS: imposta l'altezza a h  
    }  
  
    public int area() {  
        // EFFECTS: calcola larghezza*altezza  
    }  
  
}
```

Un **quadrato** è un **caso particolare** di **rettangolo** che ha larghezza sempre uguale ad altezza... **sottoclasse**!

Altro esempio: Rettangolo e Quadrato

```
public class Rectangle {  
  
    private int width;  
    private int height;  
  
    public void setWidth(int w) {  
        if (w>=0) width = w;  
    }  
    public void setHeight(int h) {  
        if (h>=0) height = h;  
    }  
  
    public int area() {  
        return width*height;  
    }  
}
```

```
public class Square extends Rectangle {  
  
    @Override  
    public void setWidth(int w) {  
        // aggiorna entrambe le dimensioni  
        if (w>=0) {width = w; height = w; }  
    }  
  
    @Override  
    public void setHeight(int h) {  
        // aggiorna entrambe le dimensioni  
        if (h>=0) { height = h; width = h; }  
    }  
  
    // public int area()    EREDITATO  
  
}
```

NOTA: `@Override` è una annotazione (facoltativa) che aumenta la leggibilità del codice e consente al compilatore di fare qualche controllo in più (ad es. che stiamo effettivamente facendo un overriding)

Altro esempio: Rettangolo e Quadrato

Esempio di contesto d'uso:

```
Rettangolo r = new Rectangle();  
r.setWidth(3);  
r.setHeight(2);  
System.out.println("AREA: " + r.area()); // AREA: 6
```

```
Rettangolo r = new Square();  
r.setWidth(3); // imposta height e width a 3  
r.setHeight(2); // imposta height e width a 2  
System.out.println("AREA: " + r.area()); // AREA: 4
```

Il comportamento è diverso!

La sottoclasse **Square**,
sebbene sia corretta dal
punto di vista dei tipi, **non
soddisfa** il principio di
sostituzione di Liskov



Altro esempio: Rettangolo e Quadrato

Dove sta il problema?

- La sottoclasse **Square** non soddisfa le specifiche della superclasse **Rectangle**, così violando il principio di Liskov

```
public class Rectangle {  
  
    public void setWidth(int w) {  
        // REQUIRES: w non negativo  
        // EFFECTS: imposta la larghezza a w  
    }  
  
    public void setHeight(int h) {  
        // REQUIRES: h non negativo  
        // EFFECTS: imposta l'altezza ad h  
    }  
  
    public int area() {  
        // EFFECTS: calcola larghezza*altezza  
    }  
  
}
```

L'effetto di **setWidth** e **setHeight** di **Square** è di cambiare sia la larghezza sia l'altezza

Altro esempio: Rettangolo e Quadrato

Possibile soluzione: interfaccia Shape (più generica)

```
public interface Shape {  
    public int area();  
    // EFFECTS: calcola l'area della forma  
}
```

```
public class Rectangle implements Shape {  
    public void setWidth(int w) {  
        // REQUIRES: w non negativo  
        // EFFECTS: imposta la larghezza a w  
    }  
    public void setHeight(int h) {  
        // REQUIRES: h non negativo  
        // EFFECTS: imposta l'altezza ad h  
    }  
    public int area() {  
        // EFFECTS: calcola larghezza*altezza  
    }  
}
```

```
public class Square implements Shape {  
    // non serve l'altezza...  
    public void setWidth(int w) {  
        // REQUIRES: w non negativo  
        // EFFECTS: imposta la larghezza a w  
    }  
    public int area() {  
        // EFFECTS: calcola larghezza^2  
    }  
}
```

Principio di Sostituzione di Liskov

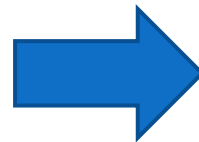
(Liskov Substitution Principle - LSP)

Il principio di sostituzione di Liskov va oltre il semplice concetto di sottotipo sostituibile per subsumption

- ▶ Esprime una **relazione tra i comportamenti** degli oggetti dei due tipi
- ▶ La relazione deve valere **per tutti i possibili contesti** in cui la sostituzione può avvenire

Problemi:

- ▶ I contesti possibili sono **infiniti**
- ▶ I contesti sono **programmi**



Verificare se il principio di sostituzione di Liskov vale tra due classi è un **problema indecidibile**

- il compilatore non può farlo...

Regole indotte dal Principio di Sostituzione

Il principio di sostituzione di Liskov nella pratica si traduce in un **insieme di regole** da seguire:

- ▶ **la regola della segnatura**

- ▶ gli oggetti del sotto-tipo devono avere tutti i metodi del supertipo
- ▶ le signature (firme) dei metodi del sottotipo devono essere compatibili con le signature dei corrispondenti metodi del supertipo

- ▶ **la regola dei metodi**

- ▶ le chiamate dei metodi del sottotipo devono comportarsi come le chiamate dei corrispondenti metodi del supertipo

- ▶ **la regola delle proprietà**

- ▶ il sottotipo deve preservare tutte le proprietà che possono essere provate sugli oggetti del super-tipo

Regola sintattica
sui tipi

Regole semantiche
(sul comportamento)

Regola della segnatura

► la regola della segnatura

- gli oggetti del sottotipo devono avere tutti i metodi del supertipo
- le signature (firme) dei metodi del sottotipo devono essere compatibili con le signature dei corrispondenti metodi del supertipo

La presenza di tutti i metodi è **garantita dal compilatore** Java, grazie ai meccanismi di ereditarietà (extends)

In caso di **overriding**, il metodo nella sottoclasse:

- deve avere la **stessa firma** del metodo nella superclasse
- può sollevare **meno eccezioni**
- può avere un **return type più specifico** (sottotipo) di quello del metodo della superclasse

Se il tipo del metodo della superclasse è

$$S \rightarrow T_1$$

il tipo del metodo della sottoclasse può essere:

$$S \rightarrow T_2$$

con $T_2 \leq T_1$ **covariante**, cioè
 $T_2 \leq T_1 \Rightarrow S \rightarrow T_2 \leq S \rightarrow T_1$

Regola dei metodi

► la regola dei metodi

- le chiamate dei metodi del sottotipo devono comportarsi come le chiamate dei corrispondenti metodi del supertipo

- In generale, un sottotipo può
 - indebolire* le precondizioni e
 - rafforzare le postcondizioni dei metodi che sono riscritti
- Per avere compatibilità tra la specifica di un metodo m del supertipo e la specifica del metodo m del sottotipo, devono essere soddisfatte le seguenti regole:
 - regola della pre-condizione $pre_{super} \Rightarrow pre_{sub}$
 - regola della post-condizione $(pre_{super} \ \&\& \ post_{sub}) \Rightarrow post_{super}$

*Più un'asserzione è debole, più è generale (meno restrittiva), mentre più è forte e più è stringente

Le postcondizioni del sottotipo possono essere più stringenti, ammesso che le precondizioni del supertipo siano soddisfatte

Regola dei metodi

Regola della pre-condizione $pre_{super} \Rightarrow pre_{sub}$

- ▶ Una sottoclasse non può restringere le pre-condizioni rispetto alla superclasse
- ▶ Il chiamante che rispetta la pre-condizione del metodo nella superclasse deve essere in grado di chiamare il metodo della sottoclasse senza violare la sua pre-condizione

Esempio:

pre_{super} : x deve essere ≥ 0

pre_{sub} : x deve essere ≥ 10 : NO, qui $pre_{super} \not\Rightarrow pre_{sub}$

pre_{sub} : x deve essere ≥ -10 : OK, qui: $pre_{super} \Rightarrow pre_{sub}$

- ▶ La pre-condizione nella sottoclasse deve essere **uguale o meno restrittiva** rispetto alla superclasse

Regola dei metodi

Regola della post-condizione ($pre_{super} \ \&\& \ post_{sub}$) $\Rightarrow post_{super}$

- ▶ La post-condizione della sottoclasse può essere **più stringente** (più restrittiva) rispetto a quella della superclasse, ma deve includere tutte le garanzie della superclasse, cioè le precondizioni del supertipo devono essere soddisfatte

Vediamo nel metodo add di IntSet/FlexIntSet:

► Nella **superclasse** IntSet

```
public boolean add(int elem) throws FullSetException {  
    // REQUIRES: numero di elementi contenuti nell'insieme minore di capienza  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
}
```

► Nella **sottoclasse** FlexIntSet

```
public boolean add(int elem) {  
    // REQUIRES:  
    // EFFECTS: se elem non è presente nell'insieme lo aggiunge e restituisce true,  
    //          altrimenti restituisce false  
}
```

Dalle specifiche (REQUIRES/EFFECTS):

$\left[\begin{array}{l} pre_{super} = (size < capacity) \\ pre_{sub} = true \end{array} \right.$	$\left[\begin{array}{l} post_{super} = (retval == (elem \notin set)) \\ post_{sub} = (retval == (elem \notin set)) \end{array} \right.$
---	--

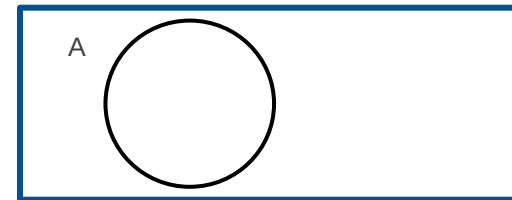
Dalle specifiche (REQUIRES/EFFECTS):

$$\left[\begin{array}{l} \mathbf{pre}_{super} = (\text{size} < \text{capacity}) \\ \mathbf{pre}_{sub} = \text{true} \end{array} \right. \left[\begin{array}{l} \mathbf{post}_{super} = (\text{retval} == (\text{elem} \notin \text{set})) \\ \mathbf{post}_{sub} = (\text{retval} == (\text{elem} \notin \text{set})) \end{array} \right.$$

Verifichiamo le regole:

► $\mathbf{pre}_{super} \Rightarrow \mathbf{pre}_{sub}$

vero, poiché $(\text{size} < \text{capacity}) \Rightarrow \text{true}$ è banalmente vera

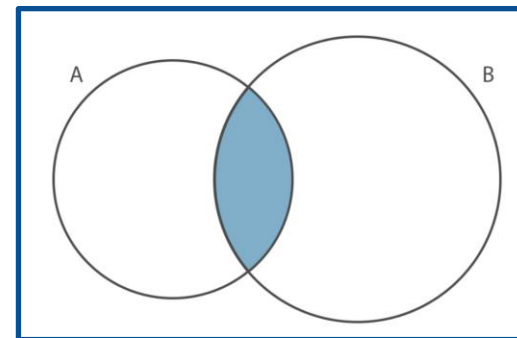


► $(\mathbf{pre}_{super} \ \&\& \ \mathbf{post}_{sub}) \Rightarrow \mathbf{post}_{super}$

vero, poiché anche

$$(\text{size} < \text{capacity}) \ \&\& \ (\text{retval} == (\text{elem} \notin \text{set})) \Rightarrow (\text{retval} == (\text{elem} \notin \text{set}))$$

è banalmente vera



(**NOTA:** il metodo della sottoclasse potrebbe fare qualcosa di diverso quando $\text{size} \geq \text{capacity}$)

Regola delle proprietà

- ▶ la regola delle proprietà

- ▶ il sottotipo deve preservare tutte le proprietà che possono essere provate sugli oggetti del supertipo

- ▶ Ogni ragionamento sulle proprietà degli oggetti basato sul supertipo è ancora valido quando gli oggetti appartengono al sottotipo
- ▶ Si tratta di proprietà degli oggetti (non proprietà dei metodi)
- ▶ Da dove vengono le proprietà degli oggetti?
 - ▶ Dal modello del tipo di dato astratto (la struttura dati che l'oggetto rappresenta)
 - ▶ Esempio: proprietà dell'insieme di interi rappresentato da un oggetto IntSet

Esempi di proprietà di oggetti

Proprietà invarianti (esempi):

- ▶ un oggetto IntSet ha sempre elementi tutti diversi tra loro
- ▶ a un oggetto FlexIntSet è sempre possibile aggiungere nuovi elementi (size < capacity)

Proprietà di evoluzione (esempi):

- ▶ il numero di elementi in un IntSet non può diminuire nel tempo
- ▶ se una chiamata add(n) restituisce true, da quel punto in poi contains(n) restituirà sempre true (stesso n)

Sono proprietà che possono essere individuate in fase di progettazione e incluse nella specifica della classe (in //OVERVIEW)

Verificare la regola delle proprietà

Per mostrare che un sotto-tipo soddisfa la regola delle proprietà dobbiamo **mostrare che preserva le proprietà del super-tipo**

Esempio: proprietà invariante di **IntSet**

- ▶ gli insiemi rappresentati hanno sempre **elementi tutti diversi** tra loro

Mostriamo che il sottotipo **FlexIntSet** verifica questa proprietà di **IntSet**...

- ▶ Insieme rappresentato come array
- ▶ Elementi diversi nell'array => elementi diversi nell'insieme rappresentato

```
public class FlexIntSet extends IntSet {  
  
    // protected int[] a; // EREDITATI  
    // protected int size;  
  
    public FlexIntSet(int capacity) { super(capacity); }  
  
    public boolean add(int elem) {  
        if (contains(elem)) {return false;} // se l'elemento è presente  
        if (size >= a.length) { // se l'array è già pieno  
            int[] tmp = new int[size + 10]; // crea array più grande  
            for (int i = 0; i < size; i++) // copia gli elementi  
                tmp[i] = a[i];  
            a = tmp; // aggiorna il riferimento  
        }  
        a[size] = elem; // aggiunge l'elemento  
        size++;  
        return true;  
    }  
  
    // public boolean contains(int elem) // EREDITATO
```

Verificare la regola delle proprietà

Per n
regol
che p

Il **costruttore** crea un array vuoto, che rappresenta un insieme vuoto

- elementi tutti diversi: ✓

disfa la
ostrare
tipo

Esem

► gl
el

Il **metodo add** aggiunge elementi solo se non presenti

- elementi tutti diversi: ✓

tSet

sempre

Mostriamo che il sottotipo **FlexIntSet** verifica questa proprietà di **IntSet**...

- Insieme rappresentato come array
- Elementi diversi nell'array => elementi

Il **metodo contains** non modifica l'array

- elementi tutti diversi: ✓

```
public class FlexIntSet extends IntSet {  
  
    // protected int[] a; // EREDITATI  
    // protected int size;  
  
    public FlexIntSet(int capacity) { super(capacity); }  
  
    public boolean add(int elem) {  
        if (contains(elem)) {return false;}  
        if (size >= a.length) {  
            int[] tmp = new int[size + 10];  
            for (int i = 0; i < size; i++)  
                tmp[i] = a[i];  
            a = tmp;  
        }  
        a[size] = elem;  
        size++;  
        return true;  
    }  
}
```

// public bool

Regola delle proprietà verificata!

EREDITATO

Invarianti e incapsulamento

La possibilità di dimostrare **proprietà invarianti** si basa principalmente sulla **proprietà di incapsulamento** della classe

- ▶ **La rappresentazione deve essere privata!** Altrimenti, tra una chiamata di metodo e l'altra, un esterno potrebbe modificarla violando l'invariante
- ▶ Se l'array che rappresenta l'insieme fosse pubblico, dall'esterno si potrebbero, ad esempio, aggiungere due copie dello stesso elemento (accedendo direttamente all'array)

Invarianti e incapsulamento



ATTENZIONE!



Quando lavoriamo con **variabili d'istanza di tipo non primitivo** (oggetti) è comune commettere un **errore di programmazione** che porta a **esporre la rappresentazione**, anche se privata

Esempio: aggiungiamo un paio di metodi a IntSet...

```
public class IntSet {  
  
    private int[] a;  
    private int size;  
  
    public IntSet(int capacity) { ... }  
  
    public int[] getElements() { return a; } // nuovo metodo  
    public int getSize() { return size; }    // nuovo metodo  
  
    public boolean add(int elem) { ... }  
    public boolean contains(int elem) { ... }  
  
}
```

Che problema hanno
questi nuovi metodi?

```
public class IntSet {  
  
    private int[] a;  
    private int size;  
  
    public IntSet(int capacity) { ... }  
  
    public int[] getElements() { return a; } // nuovo metodo  
    public int getSize() { return size; } // nuovo metodo  
  
    public boolean add(int elem) { .. }  
    public boolean contains(int elem) { ... }  
  
}
```

Che problema hanno questi nuovi metodi?

Il metodo `getElements` restituisce un riferimento all'array (che è un oggetto) dando la possibilità al chiamante di modificarlo, anche se dovrebbe essere privato!

Errore grave che viola l'incapsulamento! Non posso più assumere che siano solo i metodi a modificare la struttura